Bash Scripting

Session 9

Agenda:

01 Conditions & Loops

02 Functions

03 Intro to Crontabs

Quick Recap:

Terminal & Shell & Kernel

Environment in Linux

how to execute & run a script

Shell Variables

Positional Parameters

Read command

' ' and " "

Arithmetic Expansion

01

Conditions & Loops

if condition

If condition used to run some code only when a condition (a test) is true

basic syntax:

Method 1

```
if [[ condition ]]
then
   # some code to
do
fi
```

Method 2

```
if [[ condition ]] ; then
   # some code to do
fi
```

if condition

If else if syntax

```
If [[ condition ]] ; then
    # some stuff to do
else
    # other stuff to do
fi
```

If else if syntax

```
if [[ condition ]] ; then
    # some stuff to do
elif [[[ condition ]] ; then
    # other stuff to do
fi
```

Conditions

A test we make to execute some commands based on it.

Conditional expressions should be placed inside square braces [[Conditions]] with **spaces around them.**

Conditions

What we will discuss in Conditions ?

- Comparing String Variables
- Comparing Numerical Variables
- File Conditions

Comparing String Variables

Expression in C	Expression in BASH	Description
a == b	$a = $b or $a == $b	Checks if **a** is equal to **b**
a != b	$a != $b	Checks if **a** is not equal to **b**
a < b	$a < $b	Checks if **a** is less than **b**
a > b	$a > $b	Checks if **a** is greater than **b**

Comparing Numerical Variables

Expression in C	Expression in BASH	Description
a == b	$a -eq $b	Checks if a is equal to b
a != b	$a -ne $b	Checks if a is not equal to b
a < b	$a -lt $b	Checks if a is less than b
a > b	$a -gt $b	Checks if a is greater than b

Comparing Numerical Variables

Expression in C	Expression in BASH	Description
a >= b	$a -ge $b	Checks if a is **greater than** or equal to b
a <= b	$a -le $b	Checks if a is **less than** or equal to b

File Conditions

Expression in BASH	Description
-d $file	Checks if file is a directory
-f $file	Checks if the file exists and is an ordinary file (i.e., not a directory or a special file).
-e $file	Checks if file/directory exists

File Conditions

Expression in BASH	Description
-r $file	Checks if file is **readable**
-w $file	Checks if file is **writable**
-x $file	Checks if file is **executable**
-L $file	Checks if file is **Symbolic Link**

Comparing String Variables Example

Make a Bash Script that take **name** from user and output **welcome message** based on it’s name

$ welcome.sh

```
#!/bin/bash
echo "Enter your name:"
read name

if [[ $name == "Sara" ]] ; then
    echo "Welcome Back"
elif [[ $name == "Ahmed" ]] ; then
    echo "Welcome admin"
else
    echo "Welcome $name"
fi
```

Comparing Numerical Variables Example

Code a Bash script asks the user to input a **number** and checks whether the number is:

- greater than 10
- equal to 10
- less than 10

then outputs the appropriate message.

$ test10.sh

```
#!/bin/bash

read -p "Enter a number: " num
if [[ $num -gt 10 ]] ; then
    echo "greater than"
elif [[ $num -eq 10 ]] ; then
    echo "equal"
else
    echo "less than"
fi
```

What if i have 10..20 or even 50 possible values?

```
#!/bin/bash
read choice

if [[ $choice == "start" ]] ; then
    echo "Starting..."
elif [[ $choice == "stop" ]] ; then
    echo "Stopping..."
elif [[ $choice == "restart" ]] ; then
    echo "Restarting..."
elif [[ $choice == "status" ]] ; then
    echo "Showing status..."
else
    echo "not valid"
# and more...
fi
```

Case Statements

Case Statements

They are a cleaner and more elegant way to handle multiple conditions compared to using multiple if statements.

They are especially useful when you have a variable that could have **multiple specific values.**

Case Statements Syntax

```
case expression in
    pattern1)
        # some commands to excute
        ;; # break
    pattern2)
        # other commands to excute for pattern2
        ;;
    *)
        # commands to do if any pattern matches
        ;;
esac
```

Case Statements

```
#!/bin/bash

read choice
case $choice in
    start)
    echo "starting"
    ;;
    restart)
    echo "restarting"
    ;;
    stop)
    echo "stoping"
    ;;
    *)
    echo "not valid"
    ;;
esac
```

Loops

For Loop

A for loop is used when you want to iterate over a list of items (e.g., numbers, strings, files) or just loop for some times.

Syntax:

```
for variable in list ; do
   # Command to execute
done
```

For Loop Examples

```
for i in {1..5} ; do
    echo "number: $i"
done
```

➔ For Loop Example (Another way like C++)

```
n=5
for (( i=1; i <= $n; i++ )) ; do
    echo "Number: $i"
done
```

While Loop

The while loop enables you to execute a set of commands repeatedly until some condition occurs. It is usually used when you need to manipulate the value of a variable repeatedly.

```
while [[ condition ]]
do
    # some stuff to execute
done
```

```
while [[ condition ]] ; do
    # some stuff to execute
done
```

While Loop Example

This script is uses a while loop to count from 1 to 5.

```
#!/bin/bash
count=1

while [[ $count -le 5 ]] ; do
    echo "Counter: $count"
    (( count++ ))
done
```

Hands on

Hands on

We will create a simple game where a player fights monsters.

- The player starts with **100 energy**.
- If the player chooses **fight**, they lose 20 energy.
- If the player chooses **eat**, they gain 10 energy.
- If the player chooses **rest**, they gain 5 energy.

Requirements:

1. Take the player's name when running the script.
2. Declare and assign:
   a. A variable for the maximum energy.
   b. A variable for the current energy.
3. Display a menu with these choices:
   a. Fight
   b. Eat
   c. Rest
   d. Exit
4. Check the user's choice and perform the correct action.
5. If the current energy becomes less than or equal to 0, t
   player dies and the game ends.

Hands on

We will create a simple game where a player fights monsters.

- The player starts with **100 energy**.
- If the player chooses **fight**, they lose 20 energy.
- If the player chooses **eat**, they gain 10 energy.
- If the player chooses **rest**, they gain 5 energy.

Requirements:

1. Take the player's name when running the script.
2. Declare and assign:
   a. A variable for the maximum energy.
   b. A variable for the current energy.
3. Display a menu with these choices:
   a. Fight
   b. Eat
   c. Rest
   d. Exit
4. Check the user's choice and perform the correct action.
5. If the current energy becomes less than or equal to 0, t
   player dies and the game ends.

Hands on

```bash
#!/bin/bash

MAX_ENERGY=100
energy=$MAX_ENERGY
player=$1

echo "Welcome $player!"
echo "Energy: $energy"

while [[ $energy -gt 0 ]]
do
    echo ""
    echo "Choose an action:"
    echo "1) Fight"
    echo "2) Eat"
    echo "3) Rest"
    echo "4) Exit"

    read choice
```

```bash
case $choice in
    1)
        echo "You fought a monster!"
        energy=$((energy - 20))
        ;;
    2)
        echo "You ate food."
        energy=$((energy + 10))
        ;;
    3)
        echo "You rested."
        energy=$((energy + 5))
        ;;
    4)
        echo "Goodbye!"
        break
        ;;
    *)
        echo "Invalid option"
        ;;
esac
```

Hands on

```bash
#!/bin/bash

MAX_ENERGY=100
energy=$MAX_ENERGY
player=$1

echo "Welcome $player!"
echo "Energy: $energy"

while [[ $energy -gt 0 ]]
do
    echo ""
    echo "Choose an action:"
    echo "1) Fight"
    echo "2) Eat"
    echo "3) Rest"
    echo "4) Exit"

    read choice
```

```bash
case $choice in
    1)
        echo "You fought a monster!"
        energy=$((energy - 20))
        ;;
    2)
        echo "You ate food."
        energy=$((energy + 10))
        ;;
    3)
        echo "You rested."
        energy=$((energy + 5))
        ;;
    4)
        echo "Goodbye!"
        break
        ;;
    *)
        echo "Invalid option"
        ;;
esac
```

Hands on

```
    if [[ $energy -gt $MAX_ENERGY ]]; then
        energy=$MAX_ENERGY
    fi

    if [[ $energy -le 0 ]]; then
        echo "Game Over!"
       break
    fi

    echo "Current energy: $energy"
done
```

Functions

Functions

02

Functions in shell scripts allow you encapsulate a block of code that can be reused throughout your script.

Functions can take arguments and return values, making them powerful tools for structuring scripts.

Syntax

```
function NAME # Function Definition
{
    # code
}
NAME # Function Call
```

another way

```
NAME() # Function Definition
{
    # code
}

NAME # Function Call
```

$ hello.sh

```
#!/bin/bash
hello()
{
    for i in seq 1 3 ; do
        echo "Hello !"
    done
}

hello
```

Arguments of Function

You can passing argument by write them after function call
To use the arguments as variables, you can access their values by using $n where n is the order of the argument passed to the function, ex

- $1 first argument
- $2 second argument
- $3 third argument
- $# number of arguments

$ add.sh

```
#!/bin/bash
function add
{
    echo "$1 + $2 = $(($1 + $2))"
}

add 7 3
```

Intro to Crontabs

03

Cron

Cron is a tool that runs commands automatically at a specific time.🕑

Example:

- Run a backup every day at 3 AM
- Print something every minute
- Run a script every Sunday

Cron

Commands :

- crontab -e → Edit your crontab file, or create one if it doesn't already exist.
- crontab -l → Display your crontab file.
- crontab -r → Remove your crontab file.

Cron

Commands :

- crontab -e → Edit your crontab file, or create one if it doesn't already exist.
- crontab -l → Display your crontab file.
- crontab -r → Remove your crontab file.

Cron

```
* * * * * command
| | | | |
| | | | └── Day of week  (0–7)  Sunday = 0
| | | └──── Month       (1–12)
| | └────── Day of month   (1–31)
| └──────── Hour          (0–23)
└────────── Minute        (0–59)
```

Field	value	Description
minute	0-59	The exact minute that the command sequence executes.
hour	0-23	The hour of the day that the command sequence executes.
day	1-31	The day of the month that the command sequence executes.
month	1-12	The month of the year that the command sequence executes.
weekday	0-6	The day of the week that the command sequence executes (Sunday = 0, Monday = 1, Tuesday = 2, and so forth).

`*` -> everything

`8-11` -> from 8 to 11

`1,3,5` -> only 1 and 3 and 5

`*/2` -> every 2

Power of Scripting !

Thank You!